

T03 - Fleet & Scenario Data Engine

Completion report | Fleet Transition Planner (CSD3401 / GAM300) | Revised 22 September 2026

Deliverable	T03 - Fleet & Scenario Data Engine (M1 contract: required)
Owner	Tan Wei Jun
Supports	F01, F02, F03, F04, F05, F07
Status	Merged to main in pull request #15. Three follow-up commits are pushed on the branch and await review.
Branch	Fleet-and-Scenario-Data-Engine at 7442344; main at 8f31827
Merged commits	0a385dd - T03: Fleet & Scenario Data Engine 338945a - T03: prove fleet and plans survive save/reopen
Since the merge	5b26367 - Use one serializer and stop inventing a legacy fleet 69ae110 - Vehicles are placed objects: presets are prefabs, the fleet is the depot 7442344 - A new depot starts empty
Change size	Merged: 38 files changed, 2,034 insertions, 633 deletions. Since the merge: 37 files changed, 1,026 insertions, 678 deletions.
Verification	pnpm verify passes - 163 client tests / 21 files, 11 UI / 2 files, 1 server / 1 file; typecheck and all production builds green

Summary

T03 owns the canonical domain engine for fleet vehicles, preset references and scenario transition plans. The single transition rule - a vehicle runs on its current preset before its transition year and on the scenario's target preset from that year onward - lives in exactly one module, and the 3D preview, timeline and comparison preview all read it instead of each keeping their own copy.

Since the first issue of this report the engine was merged into main, and then changed in one significant way: **the fleet moved from the project to the depot**. A vehicle preset is a reusable type shared by every depot in the project, and placing one instantiates a real vehicle in the depot being edited. The placed object *is* the vehicle - its id, name and preset come from the object and its planning data rides along in `SceneObject.vehicle` - so there is no second list to keep in step with what stands in the world. This contradicts the M1 integration contract as written, so both contract documents were updated in the same commit rather than left to drift, and the change needs review from T01, T05 and T06.

Two corrections landed alongside it. The project file reader is now a single implementation in T01's serializer rather than two that disagreed, and nothing invents a fleet the user never entered: not a new project, not a new depot, and not a migrated legacy one.

1. What changed since the first issue

The audit in sections 2 to 9 has been brought up to the current branch. This section is the delta for a reader who has already seen the 22 September issue.

Change	Why	Where
The fleet belongs to the depot, not the project	A preset, a fleet vehicle and a 3D object were three unrelated things. Placing a preset made decorative scenery with no fleet identity, and Add vehicle made a planning record with no 3D presence, so "the fleet" and "what is in the depot" were two lists that could disagree.	domain/worldFleet.ts (new), scene/types.ts, state/fleetStore.ts, state/sceneStore.ts

Change	Why	Where
One reader for stored files	T01's serializer and T03's toM1ProjectDocument both read and migrated project files, and they disagreed: the serializer reconstructed a hard-coded six-van fleet on a version 2 migration where T03 returned an empty one. The serializer's behaviour shipped, because the repository normalizes before the store does.	project/serialization.ts is the single reader; domain/projectDocument.ts is now writer-only
Nothing invents a fleet	LEGACY_FLEET presented six invented vans as the user's own data, and legacyDefaultPlans guessed a transition year and an electric target for each. Both are gone. Legacy scenario plans naming vehicles a migrated project no longer has are pruned, because the serializer is throw-only and would otherwise fail to open every existing version 2 project on a dangling reference.	project/serialization.ts
A new depot starts empty	createDocument placed a hard-coded "sample" van in every new world and createState pre-selected it - scaffolding from the 3D starter template. Once a depot is meant to be its fleet, a van standing in the depot that is not in Fleet Management reads as a bug.	state/sceneStore.ts
Document versions moved	Project document version 4 keeps only what every depot shares: the preset library and the analysis settings. World document version 4 adds per-object vehicle data. Reopening a version 3 project moves its vehicles into the depot that was active, preserving ids so scenario plans keep resolving.	domain/contracts.ts, scene/types.ts, project/serialization.ts
Vehicle edits are undoable	Placing, editing and deleting a vehicle are ordinary scene edits now, so scene undo covers them. The previous split-store design could not offer that. Presets remain project data and stay outside scene undo.	state/sceneStore.ts, state/fleetStore.ts
SimulationInput carries the fleet	The vehicles are no longer inside the project document, so T05's input pairs the project document with the depot's vehicles explicitly. Free to change today: T05 does not exist yet and nothing consumed it.	domain/contracts.ts, domain/fleetCommands.ts

2. Responsibilities audit

Each row is a responsibility as written in docs/tech-tasks/T03 - Fleet and Scenario Data Engine.md.

#	Responsibility (verbatim)	Status	Where implemented
1	Define/own the M1 domain contracts for FleetVehicle, preset references and vehicle transition plans	Done	domain/contracts.ts (VehicleData plus FleetVehicle), scene/types.ts (SceneObject.vehicle), vehicles/types.ts
2	Replace MockVehicle as the authoritative fleet source	Done	The fleet is derived from the active depot's objects; MockVehicle and initialVehicles are deleted
3	Provide real fleet CRUD/domain operations and stable vehicle IDs	Done	placeVehicleFromPreset / updateFleetVehicle / deleteFleetVehicle; ids are minted by the scene store and immutable under patch

#	Responsibility (verbatim)	Status	Where implemented
4	Validate vehicle -> preset and scenario -> vehicle/preset references	Done	domain/references.ts; enforced on write in fleetStore and projectStore, and on read by project/serialization.ts
5	Keep transition year/target preset scenario-owned so editing one Scenario does not mutate another	Done	domain/scenario.ts deep-copy; proven in fleetCommands.test.ts
6	Resolve the effective vehicle/preset state for a requested scenario and selected year	Done	domain/effectiveState.ts
7	Provide deterministic snapshots/events consumed by T04, T05 and the 3D/product views	Done	transitionEvents(), effectiveFleetState(), currentSimulationInput()
8	Expose serializable authoritative inputs to T01/T06; do not own browser persistence itself	Done	domain/projectDocument.ts writes; reading, validating and migrating belong to T01's serializer

Goal and M1 evidence

Requirement	Status	Evidence
Goal: canonical domain/state engine replacing mock fleet state, giving every consumer the same authoritative scenario data	Met	All four former MockVehicle consumers read T03. The fleet panel, World Objects and the viewport now show one list because it is one list.
M1 evidence: create/edit a real fleet vehicle, assign different target presets/transition years in two Scenarios, switch Scenario/year, and prove the effective state changes correctly without mutating the base fleet or the other Scenario	Met	fleetCommands.test.ts - "M1 evidence: two scenarios over one depot"
Merge gate: stable vehicle/preset references across save/reopen	Met	fleetCommands.test.ts - "save and reopen": an edited vehicle and its plan return intact, and a deleted one stays deleted

3. The fleet is the depot

This is the one change a reviewer should read before the file tables, because it moves an ownership boundary the M1 integration contract had assigned elsewhere.

The model

- A **vehicle preset** is a reusable type - a prefab - shared by every depot in the project. It stays project-owned.
- **Placing a preset instantiates a vehicle** in the depot being edited. The object carries the vehicle's id, name and preset reference; its planning data lives in `SceneObject.vehicle`.
- **Only preset placements are vehicles.** A bare catalog object stays scenery, and planning data on an object with no preset does not make one - tested, so scenery cannot smuggle a vehicle in.
- **A depot's fleet is exactly what stands in it.** Vehicles placed in one depot never appear in another, and deleting the object deletes the vehicle after a warning listing the scenario plans it takes with it, from either the fleet panel or World Objects.

Scenarios were already world-bound, so scoping the fleet to the depot collapses a two-level reference to one hop and removes the all-worlds plan sweep deletion used to need. It also fixed a live bug where deleting a vehicle in one depot silently rewrote scenarios in the others.

Contract impact

Both contract documents were updated in the same commit:

Document	Change
docs/tech/m1-integration-contract.md	Fleet vehicles moved from the project-owned list to the world-owned list. Vehicle ids are now unique within their World rather than within the project, and a vehiclePlans key resolves to a vehicle in the World that scenario is bound to. The stub-truth entry for MockVehicle is struck through as removed.
docs/tech/contracts.md	Adds a Fleet vehicle entity row, records project version 4 and world version 4 together with their legacy inputs, and states that vehicle placement, editing and deletion fall under scene undo while presets do not.

Review needed from three owners. T01 (Chew Shee Yang) for the serializer's new version 3 migration path, T06 (Brandon Koh Kai Yang) for workspace ownership and the dirty-state snapshot, and T05 (Elijah Chua Jye Kang) for the SimulationInput shape. This is the one part of the branch that is not a straightforward implementation of the contract as agreed.

4. Files added

Nine engine modules and six test files, all under apps/client/src/domain/. Line counts are current.

Engine modules

File	LOC	What it does
domain/worldFleet.ts	93	New since the merge. The depot-fleet bridge. Decides what counts as a vehicle, derives FleetVehicle records from placed objects in placement order, instantiates a preset as a vehicle with neutral planning values, and lays vehicles out clear of what is already in the depot. Also rehomes a project-wide fleet into a depot when a version 3 project is reopened, preserving ids and falling back to catalog geometry when a preset has gone.
domain/effectiveState.ts	97	The single transition rule. Answers what preset a vehicle runs in a given year under a given plan, and projects a scenario's plans into ordered timeline events. Refuses to invent a target the scenario never named, and never emits an event outside the analysis period.
domain/fleet.ts	138	Live-edit validation and construction for FleetVehicle. Enforces the contract ranges: nonnegative distances, utilisation 0-1, whole operating days up to 366, dwell hours up to 24, per-kind holding terms. Returns undefined to reject a keystroke, where the serializer throws to reject a file.
domain/references.ts	103	Reference integrity. Lists every fleet and scenario use of a preset, every scenario plan keyed by a vehicle, removes a plan entry without mutating the source, and reports all invariant violations in a project as readable messages.
domain/mockProject.ts	128	Seeds the preset catalogue a new project starts from. createMockFleet remains, but only as a fixture for tests and the sample project - a real new project starts with no vehicles.
domain/m1Fixture.ts	100	The shared SIM01 fixture the domain tests build on, kept in step with the depot-owned fleet.
domain/fleetCommands.ts	75	Edits that span stores: deleting a vehicle clears its plan entries in the depot's scenarios first, then removes the object; deleting a preset is refused while anything references it. Also assembles the SimulationInput pair for T05 - now the project document plus the depot's vehicles.
domain/scenario.ts	58	Scenario document handling: version 2 documents with vehicle plans plus charging assumptions, and deep-copy helpers so duplication is genuinely independent. The legacy version 1 upgrade moved to the serializer.
domain/projectDocument.ts	22	Writes the project-owned inputs - presets and analysis settings - into a version 4 document. Reading, validating and migrating a stored document belong to project/serialization.ts, the single reader for every persistence path.

Test files

File	LOC	Covers
domain/fleetCommands.test.ts	288	Placing from a preset, live edits and invalid-patch rejection, immutable ids, vehicle-delete cascade, blocked preset delete, depot isolation, the two-scenario M1 evidence case, save/reopen round trip, and the T05 input snapshot.
domain/fleet.test.ts	134	Every validation range and rejection case for vehicles and analysis settings, inclusive period bounds, and seed integrity (including that daily distance x operating days reproduces annual distance).
domain/effectiveState.test.ts	105	Before / at / after the transition year, absent and dangling plans, event ordering and endpoints, out-of-period suppression, and two-scenario isolation.
domain/worldFleet.test.ts	85	New since the merge. What counts as a vehicle versus scenery, deriving a vehicle from its object, defensive copying of holding terms, neutral values on placement, placement order, and rehoming a project-wide fleet into a depot including layout and a preset that has gone.
domain/references.test.ts	74	Preset and vehicle reference lookups, non-mutating removal, and every invariant message.
domain/projectDocument.test.ts	56	Version 4 writing and defensive copying. The migration cases moved to serialization.test.ts with the reader.

5. Files edited

Rewritten or substantially changed

File	Change
state/fleetStore.ts	Rewritten twice. It was six hard-coded vans in a four-field MockVehicle type; it became real FleetVehicle records; it now holds only the project-owned analysis settings plus the write path into the scene. useFleetVehicles() subscribes to the object list and derives from it, memoized - selecting a freshly mapped array would hand the store a new reference on every render and never settle. Invalid patches are still rejected whole.
project/serialization.ts	The single reader for every persistence path. readProjectDocument returns the document plus any displaced fleet, so a version 3 project yields its vehicles for the workspace to place into the active world. Version 2 documents reopen with no fleet. World documents accept version 3 and read version 4 per-object vehicle data. Legacy scenario plans are pruned against the fleet that survived migration; an authoritative version 2 scenario with a dangling plan is still corruption and still throws.
state/sceneStore.ts	Gained insertObject (adds an object the domain layer built, as one undoable edit), updateVehicleData and updateObjectPreset. createDocument now returns an empty depot and createEditorState selects nothing.
state/projectStore.ts	The project snapshot no longer carries a fleet, because vehicles ride inside each world document and the scene snapshot already covers them. Repository records arrive validated and migrated, so the store no longer re-reads them. Plan writes validate against the active depot's vehicles.
components/FleetManagementPanel.tsx	The Add vehicle button is gone - placing a preset is the create path. Each row selects its vehicle in the depot, and delete warns that the 3D model goes with it and names the scenario plans that lose their entry. The empty state points at the preset panel.
components/VehiclePresets.tsx	Add to Scene became Place in depot, and it creates a real vehicle rather than scenery. The instance count reads as vehicles in this depot.
components/WorldObjects.tsx	Deleting a selected object now asks first when that object is a vehicle, listing the same affected plans as the fleet panel, and routes through deleteFleetVehicle. Plain scenery still deletes straight away.
project/portableProject.ts	Export/import moved to project version 4 and world version 4, with the legacy versions still importable.
scene/types.ts	SceneObject gained optional presetId and vehicle; SCENE_DOCUMENT_VERSION is 4.

File	Change
domain/contracts.ts	Split FleetVehicle into VehicleData - what the object carries - plus identity taken from the object. M1ProjectDocument dropped fleetVehicles and moved to version 4; SimulationInput gained fleetVehicles.

Smaller supporting edits

File	Change
components/TimelineControl.tsx, App.tsx	Both read the depot's fleet rather than a project fleet. The transition rule and the analysis period are still read from T03, never recomputed.
project/sampleProjects.ts	The sample project places its vehicles in the depot and keeps explicit transition decisions, so the demo still shows transitions now that nothing is invented.
domain/m1Fixture.ts, project/repository.test.ts, state/sceneStore.test.ts, App.test.tsx	Tests that used the hard-coded "sample" van as a ready-made fixture now place their own, which also makes what each one is editing explicit.
vehicles/types.ts, state/presetStore.ts, project/types.ts, project/analysisPeriod.ts	Unchanged since the merge. Presets carry range, maintenance, charging efficiency and owned/leased acquisition terms; START_YEAR / END_YEAR still derive from defaultAnalysisSettings.

Tests updated

App.test.tsx, ProjectControls.test.tsx, TimelineControl.test.tsx, comparisonModel.test.ts, contracts.test.ts, portableProject.test.ts, repository.test.ts, sceneStore.test.ts, projectStore.test.ts and serialization.test.ts. serialization.test.ts gained the legacy-plan pruning cases, including that an authoritative version 2 scenario with a dangling plan still throws.

6. Files and exports deleted

File / export	Why
vehicles/defaults.json vehicles/defaults.ts vite/savePresets.ts	Removed at the merge, on the owner's instruction: the preset seed file, its loader, and the dev-only POST endpoint that let the running browser rewrite the seed file on disk. The preset library Export / Import buttons and the PresetFile format went with them. Project-level .fleetproject export/import is unaffected.
toM1ProjectDocument() toM1ScenarioDocument()	T03's duplicate reader. Reading a stored file is the serializer's job; keeping two implementations produced two different answers for the same legacy project.
LEGACY_FLEET legacyDefaultPlans	Invented six vans and a transition plan for each when migrating a version 2 project, and presented them as the user's own data.
duplicateVehicle()	Superseded. Placing a preset is the create path now, and duplicating a placed object without deciding where it stands is not meaningful. This closes an open item from the first issue rather than leaving it for F02.

7. Behaviour changes a reviewer will notice

Change	Detail
A new project has presets but no vehicles	Most likely to surprise a teammate. The preset catalogue is seeded because presets are reusable types a user picks from and a vehicle cannot be created without one. Which vehicles a company runs is the user's own data, so the fleet starts empty - and so does the viewport.
Placing a preset adds a vehicle	Place in depot puts the model in the viewport and the vehicle in Fleet Management in one action. There is no separate Add vehicle button, and the two panels can no longer disagree.
Vehicles are per-depot	A vehicle placed in one depot does not appear in another, and switching depots switches the fleet.

Change	Detail
Vehicle edits undo	Placing, editing and deleting a vehicle are scene edits, so scene undo covers them. Preset edits still do not undo; presets are project data.
Deletion states its consequences, from either panel	Deleting a vehicle warns which scenario plans go with it and that the 3D model goes too, whether it starts in Fleet Management or in World Objects. Deleting a preset in use is still refused with the list of references to reassign first.
Reopening an older project still works	A version 3 project's vehicles move into the depot that was active, laid out in a row, with ids preserved so scenario plans keep resolving. A version 2 project reopens with its presets and no fleet, and its plans for vehicles that no longer exist are dropped rather than failing the open.
Invalid edits are rejected whole	Unchanged. A patch that would make a vehicle invalid leaves the previous valid record in place, per the contract's rule that invalid drafts never replace valid domain values.

8. Audit findings, gaps and boundaries

Found and fixed

- **No test proved the fleet survived save/reopen.** Found at the first audit. The implementation was correct but nothing asserted it. Round-trip tests were added in 338945a and both passed on the first run, confirming an evidence gap rather than a defect. They now cover the depot-owned fleet.
- **Two readers disagreed about legacy projects.** The serializer reconstructed a hard-coded fleet where T03 returned an empty one, and the serializer's answer was the one that shipped, because the repository normalizes before the store does. Fixed by deleting T03's reader rather than reconciling the two.
- **Deleting a vehicle rewrote other depots' scenarios.** A live bug, removed as a consequence of scoping the fleet to the depot: there is no longer an all-worlds plan sweep to get wrong.
- **A sample van stood in every new depot.** Starter-template scaffolding that had become a visible contradiction between the viewport and Fleet Management.

Open items inside T03

Item	Assessment
findReferenceIssues() is exported and fully tested but not called from product code	Intentional. It is the whole-project invariant checker; the serializer rejects or prunes bad records on the way in, so no code path can currently produce a violation. It is ready for T01/T06 to call at a persistence boundary.
A stale comment in state/projectStore.ts calls a migrated record "a version 3 document"	Cosmetic, introduced by 69ae110. It is version 4. Worth correcting in the review pass; it changes no behaviour and did not justify a commit of its own.
Vehicle layout on placement is a single row along X	Each new vehicle lands clear of whatever is already in the depot. Adequate for M1; per-scenario bay assignment is documented as future depot-editor scope.

Outside T03 - other owners

Item	Owner	Note
Fleet panel only exposes annual distance and current preset for editing	F02 (Jarrel Tay Wee Han)	The domain supports every FleetVehicle field; the remaining form controls are F02's scope.
No UI for analysis settings (period, currency, fuel price, emissions factors)	F04 (Elijah Chua Jye Kang)	The store action updateAnalysis() exists and clamps stranded replacement years.
No UI for scenario charging assumptions	F04	Scenario documents carry them at version 2 with defaults (depot charging, 0.30/kWh).

Item	Owner	Note
timelineStore.ts still uses fixed START_YEAR / END_YEAR	T04 (Jarrel Tay Wee Han)	The contract assigns year clamping and playback to T04. The constants derive from defaultAnalysisSettings so there is a single source of the numbers, and the panels already read the live period.
Bay assignment within a depot	F05 / depot editor (Chew Shee Yang)	Vehicles now have 3D identity and a position, which is what bay assignment needs. T03 would own the assignment records when it lands.

Shared files touched outside T03's column

The M1 integration contract asks owners to request small composition changes rather than restructure shared files. The follow-up commits went further than the merged work did, so the affected owners should review before this branch merges:

File	Owner	What T03 changed
project/serialization.ts	T01 (Chew Shee Yang)	Version 3 project migration with fleet rehoming, world version 4 vehicle data, and legacy plan pruning.
state/projectStore.ts state/sceneStore.ts	T06 (Brandon Koh Kai Yang)	Fleet removed from the project snapshot; three vehicle-aware scene actions; new worlds start empty.
domain/contracts.ts (SimulationInput)	T05 (Elijah Chua Jye Kang)	fleetVehicles added beside the project document. Nothing consumes it yet.

9. Verification

Check	Result
pnpm verify (typecheck + tests + builds, all packages)	Pass
apps/client tests	163 passed / 21 files
packages/ui tests	11 passed / 2 files
apps/server tests	1 passed / 1 file
TypeScript	tsc -b clean, strict with noUnusedLocals and noUnusedParameters
Builds	ui, ui-showcase, server and client production builds all succeed
Manual check (69ae110)	Placing a preset adds it to the viewport and the fleet panel; a second depot starts empty; switching back restores the first depot's vehicle.
Manual check (7442344)	A fresh project opens with an empty viewport and an empty fleet.

Known flake, not a regression: the single apps/server test can time out during a parallel pnpm verify run purely from resource contention with the jsdom client suite. It passes in isolation in well under a second, and it did not occur on the run recorded above.

Revised 22 September 2026 from branch Fleet-and-Scenario-Data-Engine at commit 7442344. The branch is pushed; main is at 8f31827 and already contains the merged half of this work.